Smart Parking Monitoring System

1. Name and Roll No. of Members:

DHRUV YADAV

2. Introduction:

Background:

Parking management has become a major issue in modern campuses, malls, offices, and urban areas. Drivers often spend a large amount of time searching for empty parking slots, which increases fuel consumption, traffic congestion, and frustration. Traditional parking systems lack real-time monitoring and efficient slot management.

The Internet of Things (IoT) provides an efficient solution by enabling smart sensing, real-time communication, and automated monitoring of parking spaces using sensors and embedded systems.

Problem Statement:

The following problems are commonly observed in traditional parking systems:

- Drivers cannot identify vacant parking slots quickly.
- Parking areas are monitored manually.
- Traffic congestion increases due to unnecessary vehicle movement.
- No real-time slot availability information is available.
- Security staff cannot efficiently manage large parking spaces.
- No centralized monitoring system exists for parking status.

Objective:

The major objectives of the Park Sense system are:

- Detect occupied and vacant parking slots in real time.
- Reduce time spent searching for parking.
- Display parking slot availability using LEDs and LCD.
- Enable wireless communication using ESP32.
- Upload parking data to cloud/dashboard for monitoring.
- Provide smart monitoring for parking administrators.
- Improve parking efficiency and reduce congestion.

3. Literature Survey:

Existing Solutions:

Existing smart parking systems generally use:

- IR sensors for vehicle detection.
- Ultrasonic sensor-based slot monitoring.
- Cloud-connected parking management systems.
- RFID-based automated parking access systems.

Research Gap:

Most existing systems are expensive and difficult to deploy on small-scale campuses or institutions. Many systems depend heavily on centralized infrastructure and lack low-cost IoT-based implementation with real-time monitoring.

Proposed Solution:

The proposed Park Sense system uses ESP32 with ultrasonic sensors to detect vehicle presence in parking slots. LEDs indicate slot availability, while LCD displays real-time parking information. Sensor data can be transmitted to a monitoring dashboard using MQTT and cloud connectivity.

Test Data (Sample Sensor Readings):

Sample Test Data:

Slot	Distance Reading (cm)	Status
P1	5	Occupied
P2	38	Empty
P3	6	Occupied
P4	41	Empty
P5	4	Occupied

Note: Distance values below threshold indicate that a vehicle is present in the slot.

4. Requirement Analysis:

Functional Requirements:

- Detect vehicles using ultrasonic sensors.
- Display slot status using LEDs.
- Show available slots on LCD display.
- Send parking data wirelessly using ESP32.
- Upload real-time data for monitoring.
- Trigger alerts when parking becomes full.

Non-Functional Requirements:

- Low power consumption.
- Real-time response.
- Wireless communication support.
- Scalable design for multiple slots.
- Reliable sensor detection.
- User-friendly monitoring system.

5. System Architecture:

System Workflow:

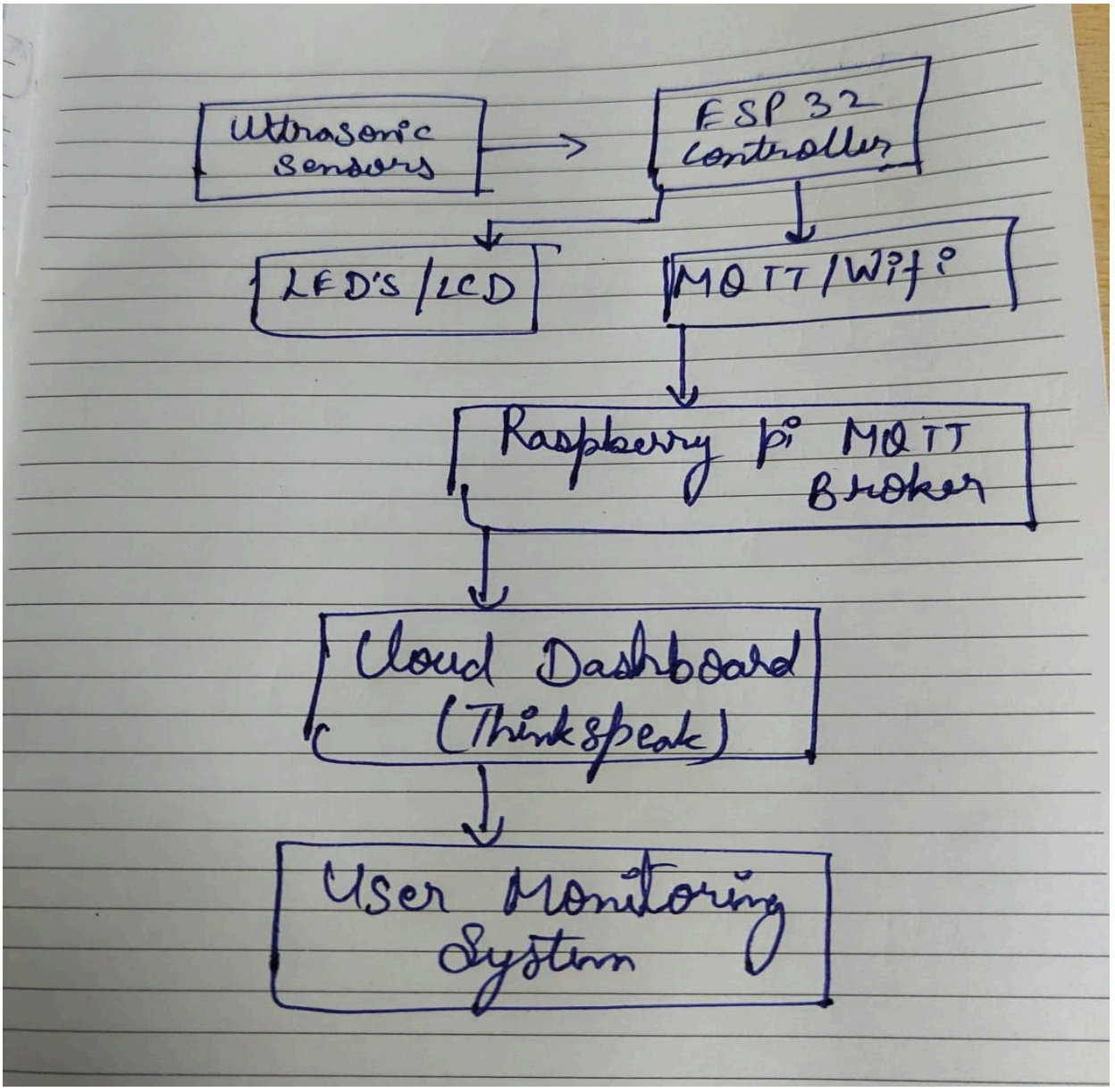
1. Ultrasonic sensors continuously monitor parking entry and exits.
2. ESP32 processes sensor readings.
3. LEDs indicate slot status:
    - Green → Entry of Vehihcle
    - Red → Exit of Vehicle
4. LCD displays parking availability.
5. Data is transmitted through MQTT/Wi-Fi.
6. Dashboard/cloud stores and visualizes parking data.

Layers of Architecture:

- Sensing Layer ---> Ultrasonic sensors detect vehicle presence.
- Communication Layer ---> Wi-Fi and MQTT protocol enable wireless communication.
- Processing Layer ----> ESP32 processes all sensor inputs and controls outputs.
- Cloud Layer ---> ThingSpeak/dashboard stores and visualizes sensor data.
- Application Layer ---> Monitoring dashboard for administrators.

Data Flow:

Sensors (HC-SR04) --> ESP32-C6 (Main Controller) --> MQTT Publish --> Raspberry Pi 4 (Mosquitto Broker + Python Uploader) --> ThingSpeak Cloud --> Web Dashboard

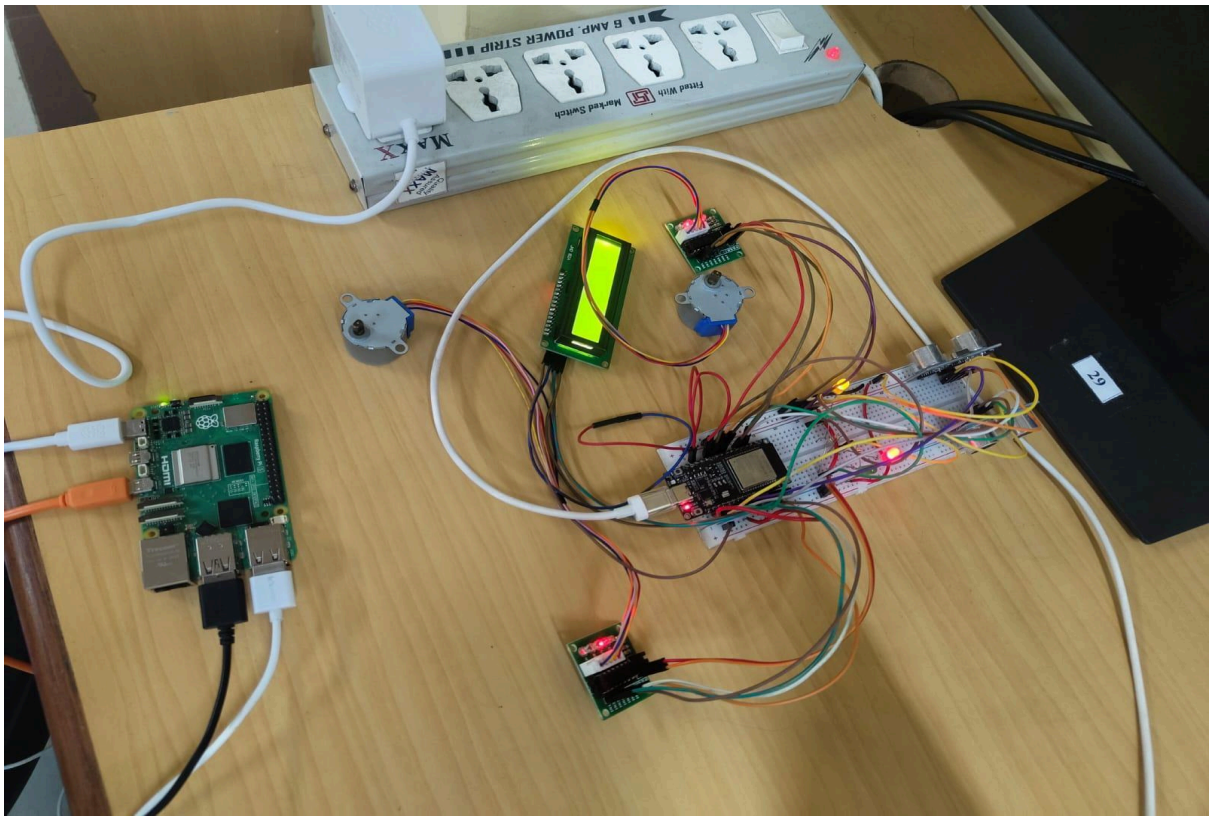


6. Hardware Design:

Components Used:

- ESP32 DevKit --> Main Controller
- HC-SR04 Ultrasonic Sensors --> Vehicle Detection
- LEDs --> Slot Status Indication
- LCD I2C Display --> Parking Information Display
- Servo Motor --> Automated Gate Control
- Raspberry Pi 4 --> MQTT Broker and Data Processing
- Power Supply --> System Power Source

Circuit Diagram:



Working Explanation:

- Ultrasonic sensors measure the distance between the sensor and nearby vehicle.
- ESP32 compares distance values with predefined thresholds.
- If a vehicle is exits: Red LED turns ON.
- Slot marked as occupied.
- If vehicle enters : Green LED turns ON.
- LCD displays the number of available slots.
- ESP32 transmits parking data to dashboard using MQTT/Wi-Fi.
- Servo motor can control automatic entry/exit gate operations.

7. Network & Connectivity:–

Communication Protocol:

The project uses three communication protocols:

- Wi-Fi (802.11 b/g/n): For Internet connectivity and cloud data upload to ThingSpeak.
- HTTP: Network protocol for node-to-cloud communication (ThingSpeak API).
- MQTT (via Eclipse Mosquitto on Raspberry Pi 4): Message broker protocol used for ESP32-C6 to Raspberry Pi communication; ESP32 publishes sensor data to topic water/Node-A1/data and Raspberry Pi subscribes and relays to ThingSpeak.

Network Configuration:

- Gateway: ESP32 acts as the primary IoT controller and manages sensor data collection and wireless communication.
- Network Topology: Star topology connecting all parking slot sensors to the ESP32 controller.
- Node-to-Cloud Communication: Via Wi-Fi --> Internet --> ThingSpeak using HTTP.
- MQTT Broker: Raspberry Pi 4 Model B running Eclipse Mosquitto on port 1883; ESP32 publishes parking slot data to MQTT topics for real-time monitoring and dashboard visualization.

Sensor --> ESP32-C6 --> MQTT Publish --> Raspberry Pi 4 (Mosquitto Broker) --> Python Uploader --> ThingSpeak (Cloud) --> Web Dashboard

.

.

.

.

.

.

.

.

.

.

.

.

.

.

.

.

8. Software Implementation:

The software implementation includes Embedded C (Arduino framework) for ESP32 firmware, Python (Paho MQTT + Requests) running on Raspberry Pi 4 for MQTT brokering and ThingSpeak upload, and optionally JavaScript for the web dashboard visualization.

Device-Side Firmware (Embedded C):

The Firmware Performs:

- Ultrasonic sensor data acquisition.
- Slot occupancy detection.
- LED and LCD control.
- MQTT communication.

Backend Logic:

- ESP32 continuously reads parking slot sensor values.
- Data is processed locally to determine occupancy.
- MQTT protocol transfers data to Raspberry Pi/dashboard.
- Dashboard visualizes slot status in real time.
- Alerts are generated when parking is full.

9. Cloud Integration:

Platform Used:

ThingSpeak (MathWorks) is used as the primary cloud platform.

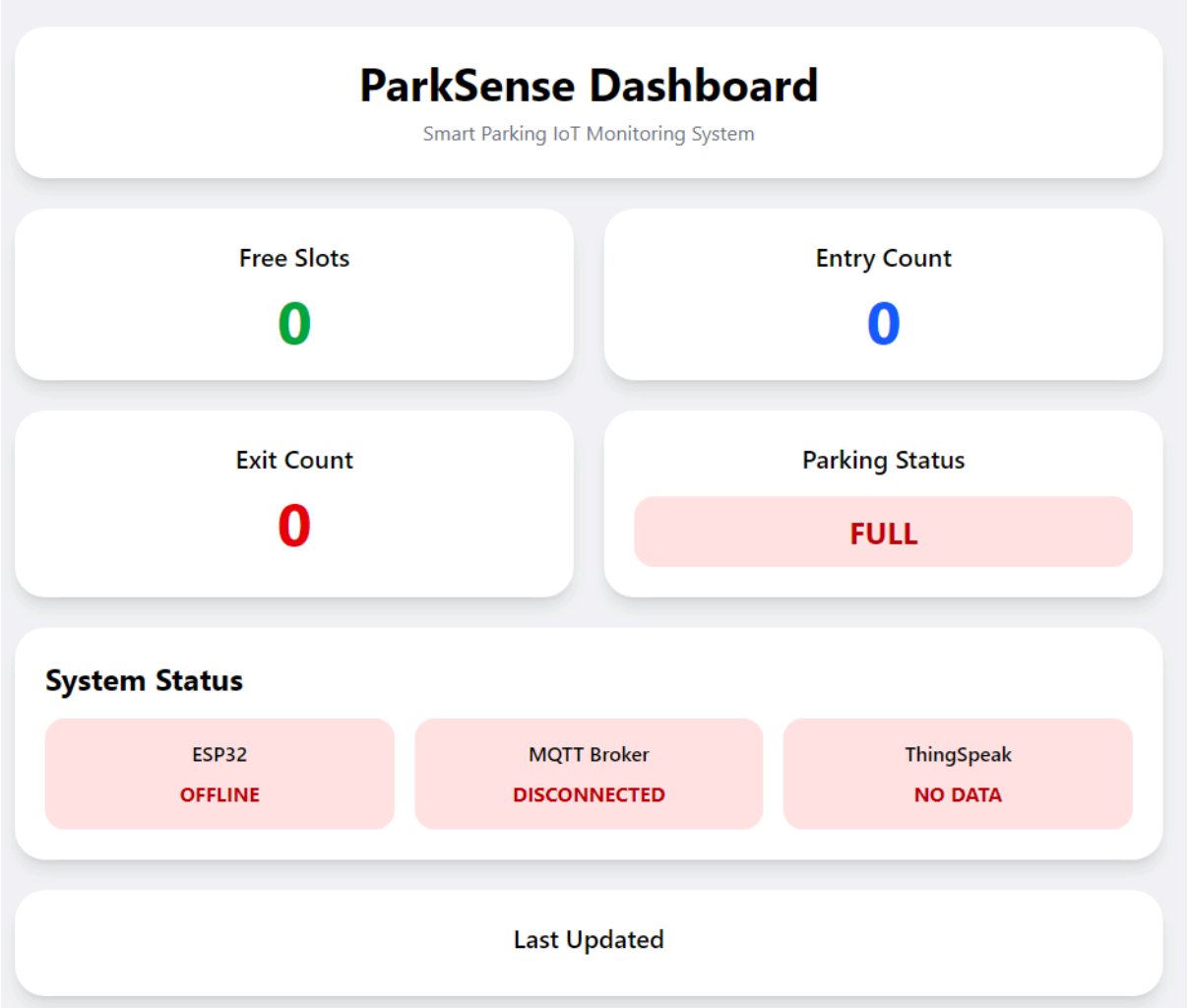
Cloud Processing:

Parking slot data is uploaded periodically to the ThingSpeak cloud platform for real-time monitoring and analysis. The dashboard visualizes parking occupancy statistics using graphs and status indicators, allowing administrators to monitor slot usage efficiently. Historical parking data can also be analyzed to observe parking trends and system performance over time.

Features of Cloud Integration:

- Real-time data synchronization across all connected clients.
- Remote accessibility from any device via web dashboard.
- visualization of historical sensor trends.
- Scalable multi-slot deployment.

10. Application / Dashboard:



The web dashboard displays real-time parking status data collected from it.

Dashboard Features (Warden View):

Dashboard Features (Administrator View): Real-time monitoring of all parking slots. Visual indication of occupied and vacant slots using status indicators. Parking occupancy statistics and slot usage monitoring. Alerts when parking slots become fully occupied. Real-time dashboard updates through MQTT communication.

Dashboard Features (Technical Staff View):

Quick identification of available parking slots. Reduced time spent searching for parking. Efficient parking guidance through live slot monitoring.

User Interface: The monitoring dashboard can be accessed through mobile phones, tablets, or desktop systems for real-time parking management and monitoring.

11. Results:

The Park Sense system successfully detected vacant parking slots using ultrasonic sensors.

Observations:

- Accurate vehicle detection achieved.
- LEDs correctly indicated slot availability.
- LCD displayed parking status successfully.
- MQTT communication functioned reliably.
- Dashboard updated parking data in real time.
- System reduced manual parking monitoring effort.

12. Testing & Validation:

Testing Performed:

- Sensor testing: TDS and HC-SR04 ultrasonic level sensor readings validated.
- Wi-Fi connectivity testing: ESP32 to ThingSpeak upload latency measured (< 5 sec confirmed).
- Cloud upload testing: data consistency between ESP32 and ThingSpeak confirmed.
- Dashboard monitoring testing: alert triggers, shows data in realtime.
- Raspberry Pi MQTT broker testing: Mosquitto broker verified using local publisher/subscriber test; end-to-end data flow from ESP32-C6 through Raspberry Pi to ThingSpeak confirmed with correct field values.

Performance Evaluation:

The system demonstrated stable communication and reliable parking slot detection. Real-time updates were successfully transmitted with low latency.

---

13. Conclusion:

The Park Sense system successfully demonstrates the implementation of IoT technology in smart parking management. The project enables real-time parking slot monitoring using ultrasonic sensors and ESP32, combined with dashboard visualization and wireless communication.

Achievements:

- Real-time parking monitoring achieved.
- Wireless IoT communication implemented.
- LCD and LED-based smart indication system deployed.
- MQTT and dashboard integration completed.
- Automated parking monitoring functionality achieved.

Limitations:

- Depends on stable Wi-Fi connectivity.
- Limited deployment scale in prototype stage.
- Environmental factors may slightly affect ultrasonic readings.

14. Future Scope:

Future improvements may include:

- AI-based parking prediction.
- Mobile application integration.
- License plate recognition.
- RFID/NFC-based authentication.
- Solar-powered parking nodes.
- Large-scale smart city deployment.
- Camera-based vehicle detection.

15. Monitoring / Maintenance:

Proper monitoring and maintenance are necessary for efficient and reliable long-term system performance.

Monitoring:

- Continuous sensor data observation via ThingSpeak and web dashboard.
- Cloud dashboard monitoring parking status in real time.
- Network connectivity checking for each ESP32 node (lastUpdated timestamp).
- Raspberry Pi MQTT broker status monitoring: verifying Mosquitto service is active and Python uploader script is running continuously.

Maintenance:

- Regular sensor calibration (HC-SR04 sensors).
- Power supply and USB connection inspection.
- Firmware updates via OTA (Over-the-Air) when available.
- Hardware inspection and troubleshooting of ESP32 and sensor connections.
- Raspberry Pi maintenance: OS updates, Mosquitto broker restarts, Python virtual environment health checks, and network IP address verification to ensure ESP32 connectivity.

Regular maintenance improves system reliability and operational efficiency.

----------END OF REPORT----------